

Project Design Phase

Proposed Solution Template

Date	23 June 2026
Name:	Varsaa Vihasini S B
Project Name:	Automated Network Request Management System
Maximum Marks:	2 Marks

Proposed Solution Blueprint

Solution Component Specification

This specification table details how your platform elements map onto the enterprise infrastructure layers.

S.No	Solution Layer	Component & Technology	Technical Description & Functional Role
01	Client Application <i>(Front-End)</i>	Service Portal & Catalog Forms <i>(HTML5, CSS3, AngularJS)</i>	Provides a unified, structured portal interface for users to input network variables (VLAN tags, subnets) while blocking formatting errors via client-side UI scripts before submission.
02	Server-Side Logic <i>(Middleware)</i>	Flow Designer Engine & Script Includes <i>(Workflow Studio)</i>	Parses inbound request metadata, dynamically evaluates approval gates against manager hierarchies, and splits parent tickets into child engineering tasks.
03	Database & Services <i>(Back-End)</i>	Relational Table Ledger <i>(u_network_database on MariaDB)</i>	Isolates active system tracking from standard data stores, logging finalized, verified configuration records permanently inside an indexed schema.
04	External Interfaces	IntegrationHub Spokes <i>(REST / SOAP)</i>	Establishes secure connectivity out to third-

S.No	Solution Layer	Component & Technology	Technical Description & Functional Role
	<i>(Integrations)</i>	<i>API</i>	party inventory registries, configuration tracking engines, and network provisioning portals.
05	Governance & Cloud <i>(Infrastructure)</i>	Multi-Instance Architecture <i>(HA Cloud PaaS)</i>	Sustains a high-availability runtime environment across mirrored data center pairs, securing files inside the attachment table (sys_attachment).

2. Comprehensive Solution Narrative

❑ Front-End Presentation Layer (Client Application View)

The system establishes a secure, web-accessible intake portal that moves users away from chaotic email threads and manual spreadsheets. Built on standard **HTML5** structures and styled with modern **CSS3 frameworks**, the interface is powered by an active **AngularJS runtime layer** native to the ServiceNow platform.

As a user populates network request configuration fields, **Catalog Client Scripts** and **UI Policies** immediately scan the data strings. If a user attempts to submit an incorrectly formatted IP subnet address or an invalid VLAN designation code, the client-side layer halts the transaction instantly. This prevents corrupted metadata from consuming processing cycles downstream.

❑ Core Process Automation (Server-Side Logic & Middleware)

Once a clean request passes client-side validation barriers, its payload is handed off asynchronously to the **Flow Designer workflow engine**. The server engine runs background check routines, querying relational user directory tables to find the appropriate management stakeholders for authorization.

The system then generates interactive approval routing tasks. Once authorizing managers verify and digitally sign off on the work card, backend **Script Includes** and **Data Brokers** run task-splitting processes. This breaks the parent request down into technical child assignment tasks, automatically distributing them to the active queues of the responsible network engineering teams.

❑ Data Persistence & Integration Engine (Back-End Storage)

The storage architecture separates operational intake data from transactional logs to maximize search indexing speeds and maintain data hygiene:

- **The Foundation Layer:** Basic login metadata, user profiles, and general asset parameters are referenced directly from the system configuration tables (backed by an underlying, enterprise-grade relational database engine).
- **The Custom Ledger Layer:** Once an engineer marks a technical network task as closed, the application executes a secure write script to store the finalized parameters inside our custom table registry (u_network_database).
- **The Attachment Repository:** Supplementary documents, design sheets, and diagnostic output logs are converted to secure binary blocks and stored within the dedicated system attachment archive (sys_attachment).

To eliminate manual data entry gaps for engineers, the backend incorporates the **IntegrationHub REST API Spoke Framework**. This out-of-the-box communication channel securely bridges the platform with external systems, third-party assets, or configuration management databases, allowing for the future automation of infrastructure changes.

3. Operational Guardrails & Architecture Characteristics

- **Unified Security Controls:** System visibility is governed by strict **Contextual Access Control Lists (ACLs)**. Requesters are limited to tracking their personal open tickets on the workspace dashboard, while full modifications are locked behind strict role privileges reserved for network operators.
- **High-Availability Scaling:** The application is deployed across a secure **Multi-Instance Enterprise Cloud Architecture**. Requests are distributed via load balancers over geographically isolated, redundant data center pairs to ensure continuous uptime and handle processing spikes of up to **10,000 concurrent records**.
- **Performance Optimization:** Through the strategic use of browser-side variable caching and carefully optimized database indexing keys, form actions and search loops resolve in **under 1.5 seconds**, preventing system lag.